

SESSION 10 - JOINS - Part 2

Task 1: Create two tables: Influencers (id, name) and Collaborations (id, influencer1_id, influencer2_id, collab_date). Write a SQL FULL JOIN query to list all influencers and show their collaboration partner names if any, including influencers with no collaborations.

Query / Answer:

```
SELECT
    i.name AS influencer,
    p.name AS partner,
    c.collab_date
FROM Influencers i
LEFT JOIN Collaborations c
    ON i.id = c.influencer1_id
LEFT JOIN Influencers p
    ON c.influencer2_id = p.id
```

UNION

```
SELECT
    i.name AS influencer,
    p.name AS partner,
    c.collab_date
FROM Influencers i
RIGHT JOIN Collaborations c
    ON i.id = c.influencer1_id
LEFT JOIN Influencers p
    ON c.influencer2_id = p.id;
```

Image / Screenshot:

The screenshot shows a SQL query editor with the following query:

```
20 from influencers i LEFT JOIN collaborations c on i.id = c.influencer1_id left join influencers p on c.influencer2_id = p.id
21
22 UNION
23
24 select i.name as first_influencers , p.name as secound_influencer , c.collab_date
25
26 from influencers i right JOIN collaborations c on i.id = c.influencer1_id left join influencers p on c.influencer2_id = p.id;
```

Below the query editor is the 'Result Grid' showing the output of the query:

	first_influencers	secound_influencer	collab_date
▶	Deep	Aarav	2026-07-01
	Aarav	Riya	2026-07-05
	Riya	NULL	NULL
	Neha	NULL	NULL
	Kabir	NULL	2026-07-08

SESSION 10 - JOINS - Part 2

--

SESSION 10 - JOINS - Part 2

Task 2: Using a SELF JOIN, write a query on a table called Playlists (id, user_id, playlist_name, parent_playlist_id) to display each playlist alongside its parent playlist name, similar to how Spotify shows nested playlists .

Hint : Join Playlists with itself on parent_playlist_id = id .

Query / Answer:

```
select l1.playlist_name , l2.playlist_name from playlists l1 LEFT join playlists l2 on l1.parent_playlist_id = l2.id;
```

Image / Screenshot:

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
17 (104, 'Sleep Playlist', 8),
18 (105, 'Favorites', NULL);
19
20 ; Using a SELF JOIN, write a query on a table called Playlists (id, user_id, playlist_name, parent_playlist_id) to display each play
21 <em><strong>Hint :</strong> Join Playlists with itself on parent_playlist_id = id .</em>
22
23 select l1.playlist_name , l2.playlist_name from playlists l1 LEFT join playlists l2 on l1.parent_playlist_id = l2.id;
```

Below the query editor, the 'Result Grid' is displayed, showing the results of the query. The grid has two columns: 'playlist_name' and 'playlist_name'. The results are as follows:

playlist_name	playlist_name
My Music	NULL
Workout Mix	My Music
Morning Run	Workout Mix
Travel Songs	NULL
Road Trip	Travel Songs
Party Hits	NULL
Dance Mix	Party Hits
Relaxing Music	NULL
Sleep Playlist	Relaxing Music
Favorites	NULL

SESSION 10 - JOINS - Part 2

Task 3: Given three tables: Users (id, username), Orders (id, user_id, order_date), and Payments (id, order_id, amount), write a SQL query using multiple JOINS to display each username, their order date, and payment amount, showing all users even if they have no orders or payments.

Query / Answer:

```
select users2.username , orders2.order_date , payments2.amount from users2 left join  
orders2 on users2.id = orders2.user_id left join payments2 on orders2.user_id =  
payments2.order_id;
```

Image / Screenshot:

username	order_date	amount
Rahul	2025-07-01	1200.00
Rahul	2025-07-10	1200.00
Priya	2025-07-05	850.50
Amit	NULL	NULL
Sneha	2025-07-08	2100.00
Karan	NULL	NULL

SESSION 10 - JOINS - Part 2

Task 4: You notice that your JOIN query between Zomato's Restaurants and Reviews tables is returning duplicate rows for some restaurants. Modify your query to eliminate duplicates and explain in one line why the duplicates were happening .< br>< br>< em>< strong>Hint :< /strong> Use DISTINCT or GROUP BY and consider the relationship between restaurants and reviews .< /em>

Query / Answer:

```
select restaurants11.restaurant_name , AVG(reviews.rating) as rating from restaurants11
inner join Reviews on restaurants11.restaurant_id = Reviews.restaurant_id group by
restaurant_name ;
```

Image / Screenshot:

The screenshot shows a SQL query editor with a query that uses an inner join between restaurants11 and Reviews tables, grouped by restaurant_name. Below the query, a 'Result Grid' displays the output with columns restaurant_name and rating. The results show four rows: Barbeque Nation (4.50000), Dominos Pizza (3.85000), KFC (4.00000), and Subway (3.80000).

restaurant_name	rating
Barbeque Nation	4.50000
Dominos Pizza	3.85000
KFC	4.00000
Subway	3.80000

SESSION 10 - JOINS - Part 2

Task 5: Write two different JOIN queries on a Products and Categories table (like Flipkart) to list all products with their category names, but use different join conditions in each. Briefly explain which join condition is more efficient and why.

Query / Answer:

```
select categories.category_name , products10.product_name , products10.price from
categories Inner join products10 on categories.category_id = products10.category_id;
select categories.category_name , products10.product_name , products10.price from
categories Inner join products10 on categories.category_id = products10.product_id;
select categories.category_name , products10.product_name , products10.price from
categories Inner join products10 on products10.category_id >= categories.category_id;
select categories.category_name , products10.product_name , products10.price from
categories Inner join products10 on categories.category_id >= products10.category_id;
```

Image / Screenshot:

The screenshot shows a SQL IDE interface. The query editor contains four SQL queries, each using a different JOIN condition to list all products with their category names. The result grid displays the output of the first query, showing columns for category_name, product_name, and price.

Query 12:

```
select categories.category_name , products10.product_name , products10.price from
categories Inner join products10 on categories.category_id = products10.category_id;
```

Query 13:

```
select categories.category_name , products10.product_name , products10.price from
categories Inner join products10 on categories.category_id = products10.product_id;
```

Query 14:

```
select categories.category_name , products10.product_name , products10.price from
categories Inner join products10 on products10.category_id >= categories.category_id;
```

Query 15:

```
select categories.category_name , products10.product_name , products10.price from
categories Inner join products10 on categories.category_id >= products10.category_id;
```

Result Grid:

category_name	product_name	price
Laptops	LG OLED C4	129999.00
Laptops	Mi Smart TV X	34999.00
Headphones	Sony WH-1000XM5	29999.00
Headphones	Boat Rockerz 450	1499.00
Headphones	Apple Watch Series 10	45999.00
Headphones	Samsung Galaxy Watch 7	31999.00
Headphones	LG OLED C4	129999.00
Headphones	Mi Smart TV X	34999.00
Smart Watches	Apple Watch Series 10	45999.00
Smart Watches	Samsung Galaxy Watch 7	31999.00
Smart Watches	LG OLED C4	129999.00
Smart Watches	Mi Smart TV X	34999.00
Televisions	LG OLED C4	129999.00